INTEGRATION, the VLSI journal 110 (2026) 102797

Contents lists available at ScienceDirect

Integration, the VLSI Journal

journal homepage: www.elsevier.com/locate/vlsi

AssertMiner-pro: Enhanced module-level spec generation and assertion mining with LLM guided by top-down hierarchical strategies☆

Yonghao Wang [a], Jiaxin Zhou [c], Hongqin Lyu [a,b], Boling Chen [d], Mingyu Shi [e], Zhiteng Chao [a], Tiancheng Wang [a,*], Huawei Li [a,b]

[a] State Key Lab of Processors, Institute of Computing Technology, Chinese Academy of Sciences, Beijing, China
[b] University of Chinese Academy of Sciences, Beijing, China
[c] Beijing Normal University, Beijing, China
[d] Beijing University of Posts and Telecommunications, Beijing, China
[e] Nanjing University, Nanjing, China

ARTICLE INFO

Keywords:
Functional verification
Assertion generation
Large language model
Specification extraction

ABSTRACT

Assertion-based verification (ABV) is an important verification technique that detects potential design errors by embedding assertions into the design. In recent years, significant progress has been made in large language models (LLMs)-based assertion generation. However, these existing approaches mainly focus on top-level modules, leaving lower-level modules insufficiently covered and errors undetected. To address this limitation, this paper proposes AssertMiner-pro, an enhanced module-level assertion generation framework extended from AssertMiner. The framework integrates abstract syntax tree (AST) analysis to extract reliable information from the design under verification, and adopts a top-down hierarchical guidance strategy to steer LLMs to systematically traverse from top-level modules to lower-level modules. This enables more accurate inference of module-level functional specifications and the generation of a larger number of fine-grained deep assertions. Experimental results demonstrate that when integrated with current SOTA methods, AssertLLM and AssertGen, AssertMiner-pro outperforms the original AssertMiner framework in terms of module-level assertion coverage and error detection capability, significantly improving verification quality and providing more comprehensive support for hardware designs. The whole implementation is shown in a repository: https://github.com/Wangsafa/AssertMiner-pro.git.

1. Introduction

Functional verification is an essential stage in the design flow of integrated circuits (ICs), where verification engineers rigorously evaluate whether the register-transfer level (RTL) implementation faithfully adheres to the architectural specifications. Even subtle functional deviations or corner-case design flaws can escalate into severe system-level failures [1], underscoring the critical importance of ensuring that IC designs are functionally correct, safe, and secure. As contemporary ICs continue to expand in scale and architectural complexity, incorporating diverse functional units and intricate interactions, the verification challenge correspondingly intensifies. Consequently, achieving high assurance in functional correctness has become a highly time-consuming endeavor [2].

Assertion-based verification (ABV) is widely adopted in RTL designs due to its ability to enhance observability and reduce simulation debugging time by up to 50% [3,4]. In particular, high-quality SystemVerilog assertions (SVA) for formal property verification (FPV) are critical within ABV methodologies [5,6], particularly when they could accurately reflect both high-level design intent and low-level RTL details. However, specifications often contain ambiguities [7,8], and different designers may interpret and implement the same specification in significantly different ways, making the translation of specifications into effective SVA assertions a time-consuming and labor-intensive process [9]. Crafting a comprehensive set of assertions further requires substantial human expertise, as an insufficient set risks inadequate coverage and allows corner-case design bugs to escape into production.

☆ This article is part of a Special issue entitled: 'CCF-DAC 2025' published in Integration, the VLSI Journal.

* Correspondence to: No. 6 Ke-Xue-Yuan South Road, Zhong-Guan-Cun, Haidian District, Beijing, 100190, China.
E-mail addresses: wangyonghao22@mails.ucas.ac.cn (Y. Wang), zhoujiaxin@mail.bnu.edu.cn (J. Zhou), lvhongqin24b@ict.ac.cn (H. Lyu), chenboling@bupt.edu.cn (B. Chen), mingyu.shi@smail.nju.edu.cn (M. Shi), chaozhiteng@ict.ac.cn (Z. Chao), wangtiancheng@ict.ac.cn (T. Wang), lihuawei@ict.ac.cn (H. Li).

https://doi.org/10.1016/j.vlsi.2026.102797
Received 18 January 2026; Received in revised form 20 May 2026; Accepted 12 June 2026
Available online 16 June 2026

These challenges highlight the need for automated and scalable techniques capable of rapidly generating high-quality assertions.

The application of large language models (LLMs) in the hardware domain has inspired new approaches to generating assertions for hardware design. Research on automatically generating SVA using LLMs can be broadly divided into two categories. The first category generates assertions using both the specification and RTL code [10–12], which helps capture more detailed functional characteristics. However, the golden RTL model is difficult to obtain in reality, and the RTL code to be verified may contain errors. Therefore, the RTL-derived assertions may inherit RTL bugs that conflict with the specification, weakening their ability to catch internal errors. The second category relies solely on LLMs to translate specification into assertions without using RTL code [13–17]. These methods are tend to miss assertions that verify implementation details present in RTL. To alleviate this problem, some studies have sought to improve assertion-generation accuracy and completeness by establishing mappings between the specification and assertions [18], and by exploring work related to assertion repair [19]. However, due to the inherently high structural similarity among assertions, this challenge remains fundamentally unresolved.

More importantly, existing methods all rely on the specification file of the top-level module to extract high-level functional logic. As a result, these methods are only capable of generating assertions for internal signals within the top-level module, while neglecting the necessity of assertions targeting low-level modules. In modern SoCs, many bugs reside in internal logic chains that are beyond the reach of top-level assertions. Capturing these bugs with top-level assertions is either difficult or challenging, especially when it comes to pinpointing the exact module or signal path where they occur. AssertMiner [17], the previous version of this framework, makes initial attempts at module-level assertion generation. However, it generates specifications for each submodule in isolation, ignoring hierarchical context, which leads to the omission of lower-level module specifications as call depth increases. In contrast, in real verification practice, engineers typically follow established IC design methodologies (e.g., Top-down [20]), validating the design layer by layer. After ensuring the functional correctness of a higher-level module, they then move down into the modules it calls to write deep assertions—namely, assertions embedded deeply into the low-level modules—that can monitor properties at a finer granularity within the module hierarchy [17]. This enables them to detect errors deeply embedded in the module hierarchy and locate error sources more quickly, thereby significantly reducing debugging time.

Inspired by human-centric IC design methodologies such as top-down verification [20] and design-for-verification (DFV) [21], we introduce an enhanced framework for mining deep assertions that explicitly monitor module-level internal signal behaviors. Unlike existing approaches that rely solely on top-level specifications, the proposed framework follows a hierarchical, top-down analysis strategy to progressively extract reliable information across different abstraction layers of the design. By systematically incorporating implementation-level details from internal modules while remaining robust to potential inconsistencies or errors in low-level implementations, the framework guides large language models to reason beyond top-level functionality and generate assertions that are deeply embedded within the module hierarchy.

The contributions of this paper are summarized as follows:

- We propose a top-down reasoning framework that enables LLMs to generate complete module-level specifications across deep design hierarchies. By leveraging top-down hierarchical guidance and RTL-derived structural cues, AssertMiner-pro systematically propagates specification intent, ensuring consistent specification generation for all modules.
- AssertMiner-pro enhances AssertMiner by providing LLMs with RTL code as a reference, enabling the capture of more comprehensive module-level information without being affected by potential errors in the RTL. This enhancement enables LLMs to capture more comprehensive information, thereby generating finer-grained deep assertions that capture detailed internal signal behaviors across multiple modules.
- Experimental results show that the generated module-level assertions by AssertMiner-pro exhibit higher structural coverage compared to AssertMiner, and when integrated with the SOTA LLM-based assertion generation methods, significantly improve error detection capability over the AssertMiner baseline.

The remainder of the article is organized as follows: Section 2 provides relevant background information. Section 3 introduces our observations and motivations derived from the experiments and investigations of the current assertion generation approach. Section 4 describes the detailed framework of AssertMiner-pro for generating module-level specifications and deep assertions. Section 5 presents the experimentation and evaluation on structural coverage improvement with deep assertions. Section 6 evaluates the error detection ratio using deep assertions via mutation testing. Finally, Section 7 summarizes the paper.

2. Background

2.1. Assertion generation based on NLP

Schema-based assertion generation [22] is one of the early attempts at automatic assertion generation through the static analysis of natural language processing (NLP). It constructs procedural models by instantiating templates with design constructs and constraints.

However, this method mainly targets combinational logic or static properties and cannot generate assertions with temporal characteristics. Therefore, subsequent studies [23–26] further explored automatic generation mechanisms for temporal assertions based on this approach. Static and dynamic analysis techniques were introduced to capture temporal dependencies among design signals, thereby enabling the automated derivation of sequential or time-constrained assertions. Meanwhile, to prevent RTL design flaws from propagating and influencing the generated assertions, [27] proposed to employ subtree analysis to generate assertions directly from the specification. Building on this foundation, Ganapathy P. et al. [28] employed sentence classification, named entity recognition, and enhanced parse tree analysis to automatically generate assertions, thereby reducing the workload of writing assertions while simultaneously lowering error rates. However, purely machine-learning methods are susceptible to issues such as long sentences and grammatical errors, leading to inaccuracies in the generated output. Consequently, Iman et al. [29] proposed a hybrid rule-based and machine-learning approach to improve the accuracy of assertion generation.

Due to the lack of semantic relevance understanding in traditional NLP methods, leveraging LLMs with stronger semantic comprehension has emerged as a promising direction for improvement in assertion generation.

2.2. Assertion generation based on LLMs

Building upon their acknowledged natural language processing capabilities, a growing body of research has begun to explore LLMs in hardware assertion generation. Rahul Kande et al. [15] were the first to investigate the feasibility of leveraging LLMs to automatically generate hardware security assertions. To explore the adaptability of LLMs in the domain of Assertion-Based Verification (ABV), Maddala K et al. [30] proposed an LLM-assisted assertion generation framework tailored for dynamic simulation scenarios. Meanwhile, Mali B. et al. [31] leveraged GPT-4 to standardize and structure design specifications, and incorporated a simulation log feedback loop to enable automated assertion generation and iterative refinement.

Considering the complexity of generating assertions, Yan et al. [13] proposed AssertLLM, which decomposes the task of generating assertions from natural language specifications into multiple subtasks. On this basis, Wu et al. [18] proposed Spec2Assertion, which employs regularization and Chain-of-Thought prompting to generate high-quality assertions directly from specifications. However, their approach still struggled to bridge the semantic gap between specifications and RTL details. To address this, Bai et al. [7] proposed AssertionForge, a knowledge-graph-based framework, linking natural language specifications with RTL semantics to improve assertion quality. Since most approaches generate assertions only at the top-module level, Lyu et al. [32] proposed to achieve multi-layer assertion generation by constructing cross-layer signal bridging.

2.3. IC design methodologies

In contemporary IC development, Top-down design [20] and Design-for-Verification (DFV) [21] have become two of the most widely adopted methodologies [33]. Both approaches serve as foundational pillars in modern design practice, contributing significantly to improved development efficiency and higher overall design quality.

Top-down. The Top-down methodology adopts a hierarchical design strategy that starts from a system-level perspective, emphasizing high-level functional and performance requirements. The design is then incrementally decomposed and refined through successive abstraction layers until it reaches the hardware implementation stage. By supporting early verification and continuous refinement throughout the design process, the Top-down flow helps reduce costly redesigns later in the cycle, encourages the reuse of existing design components, and strengthens collaboration among teams working at different abstraction levels.

Design-for-verification (DFV). DFV emphasizes improving a design's verifiability across the entire development process. Its central objective is to lower verification complexity and enhance overall verification productivity by integrating verification-oriented features directly into the design itself. This involves structuring the design into smaller, modular blocks that can be verified independently, increasing internal signal observability and controllability, and formulating test strategies and coverage objectives at an early stage of the design cycle.

3. Observations and motivations

3.1. Failure of current techniques to support hierarchical deep assertion generation

Current LLM-based assertion generation methods invariably rely on the top-level design specification as an indispensable input. However, such specifications are typically high-level and coarse-grained. This limits these methods to generating assertions that focus only on top-level behaviors. As a result, they are fundamentally incapable of generating deep assertions that monitor internal signals within lower-level modules. This limitation is inconsistent with practical IC design and verification methodologies, where internal signal-level properties are essential.

To validate the soundness of our hypothesis, we evaluate several state-of-the-art assertion generation methods. We modify their signal-extraction pipelines by replacing top-level signal names with those from deep internal modules. This forces the models to generate deep assertions. The results show that these methods struggle to produce correct or practically effective deep assertions, highlighting the intrinsic difficulty of this task. Moreover, even in the few successful cases, the resulting assertions provide nearly zero functional coverage. For example, in an I2C case study, the forced deep assertions generated by AssertLLM [13] are largely ineffective. They mainly verify trivial properties such as signal bit-width constraints and contribute negligible improvement to functional coverage, as illustrated below.

```
assert property($bits(i2c_master_byte_ctrl.ena) == 1);
```

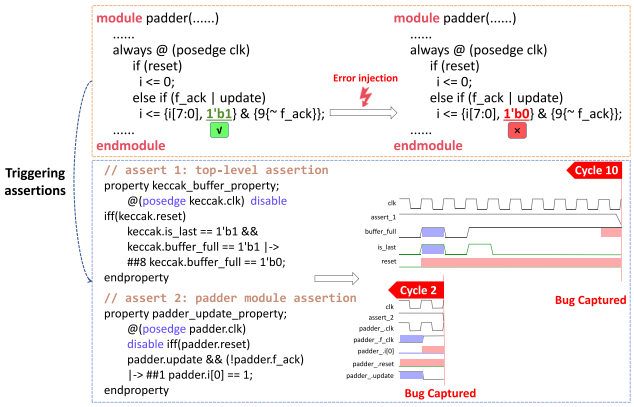

Fig. 1. A case of rapid assertion triggering and efficient debugging via assertion in the padder module of SHA3 [34].

3.2. Effectiveness of deep assertions for early error detection and localization

During our simulation experiments, we observed that when bugs occur in internal signals of deep modules, deep assertions targeting these signals can detect errors more promptly than top-level assertions. Moreover, the relationships encoded in deep assertions provide valuable information for directly localizing the fault within the module. In contrast, top-level assertions require a more cumbersome process of bug localization and debugging.

Fig. 1 presents a case study illustrating the advantages of module-level deep assertions over top-level assertions in detecting and localizing design errors. In this example, a single fault is injected into the padder module by mistakenly flipping a bit from 1 to 0. As shown in Fig. 1, this error can simultaneously trigger both a top-level assertion and a deep assertion within the affected module. The top-level assertion is activated only after ten clock cycles, and additional analysis is required to further localize the source of the error. In contrast, the module-level deep assertion is triggered within two clock cycles and directly identifies the erroneous signal, thereby significantly reducing debugging time.

3.3. Inferring designers' mental module specifications: A key challenge for deep assertion generation

n practice, IC designers can implement multiple modules from a single top-level specification. This is possible because they implicitly extract module-level specifications from the top-level description in their minds. These implicit specifications describe the functionality of each module as well as its internal signals, although they are not formally documented. Therefore, if we can develop methods to extract such implicit module specifications, we can leverage existing assertion-generation techniques to produce effective deep assertions.

While it is straightforward to generate a module specification directly from RTL code, as discussed previously, the RTL under verification may contain errors. Consequently, specifications derived directly from code could inherit these bugs, potentially reducing the effectiveness of the generated assertions for debugging.

Therefore, **a critical challenge** is how to generate accurate module-level specifications without relying on implementation details of the RTL?

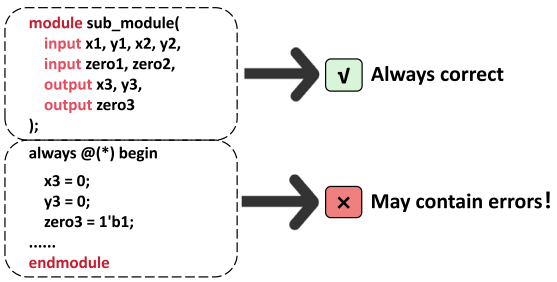

Fig. 2. Assumptions on RTL modules: correct ports and error-prone internal implementation.

3.4. Two fundamental assumptions and a important conjecture

In practical verification scenarios, designers typically perform an initial review of the RTL code delivered to verification engineers. Consequently, the RTL code rarely contains obvious errors; otherwise, the participation of verification engineers would be redundant. Based on this realistic assumption, we propose two fundamental hypotheses.

Assumption 1. Within a module's RTL design, potential bugs are typically confined to internal implementation details. In contrast, interface-related information—such as port declarations and structural connections—is generally correct, as shown in Fig. 2.

Assumption 2. Errors within a module are often minor oversights made by designers, and such small mistakes typically do not affect the verification engineers' overall understanding of the module's intended functionality.

These two assumptions are supported by numerous existing studies. For instance, in current research on RTL generation [33,35] and RTL fix [36,37], datasets typically provide LLMs or designers with correct port information and an overall description of the design as reference. This indicates that these studies are based on the premise that interface information is reliable and minor implementation errors do not affect the overall understanding of the module's intended functionality. If this premise did not hold, the methodology and conclusions of these studies would lack a foundational basis, thereby underscoring the validity of the aforementioned assumptions.

Based on these two hypotheses, we formulate **an important conjecture**: Can we extract the portions of the RTL code that are known—or highly likely—to be correct, provide this trusted information to an LLM, and leverage the LLM's strong reasoning capabilities to infer the functional behavior of the module and its internal signals?

If so, we may be able to automatically derive an accurate module-level specification that does not rely on potentially erroneous implementation details of the RTL.

4. The framework of AssertMiner-pro

4.1. Workflow overview

To address the challenge of generating deep assertions for modules that reflect the micro-architectures of a design, we propose a framework called AssertMiner-pro. The framework applies AST-based structural analysis and a top-down reasoning process integrated with the DFV methodology. It constructs dedicated specifications for each module, enabling the mining of targeted deep assertions for module-level signals. Specifically, as illustrated in Fig. 3, the framework follows a five-stage workflow for automatically generating module-level specifications and deep assertions:

- **Reliable Information Extraction:** Extract reliable structural information from the RTL code, including module hierarchy, signal dependencies, and data-flow relationships.
- **Module-Level Specification Generation:** Generate module-level specifications for each module by reasoning over the extracted structural information.
- **Verification Feature Extraction:** Decompose module specifications into fine-grained verification features that describe key functional properties.
- **Deep Assertion Generation:** Generate fine-grained deep assertions for module-level signals guided by the extracted verification features.
- **Assertion Filtering:** Remove incomplete or invalid assertions caused by insufficient or missing information in the generation process.

We provide a detailed description of each step in the following sections.

4.2. Step 1: AST-based structural analysis and reliable information extraction

In practical scenarios, based on Assumption 1, the RTL code provided to verification engineers is typically assumed to contain correct specifications of each module's input and output ports. Under this assumption, several types of reliable structural information can be identified, including the module call graph (MCG), the I/O table, the dataflow graph, and the inter-module signal chains. All of these can be directly extracted from the RTL code. Such extraction helps improve the understanding of both individual module functionality and the behavior of the system as a whole. To obtain the required static structure accurately, we use Tree-sitter to parse the RTL into an abstract syntax tree (AST). The detailed procedure is described below.

1. **Module Call Graph Extraction**: From AST instantiations, module hierarchy and interconnections are derived. An MCG is then constructed where nodes are modules and edges are instantiations.
2. **I/O Table Extraction**: Again from the AST, we extract each module's I/O declarations and resolve instantiation mappings to link ports to parent signals, producing a per-module I/O table (name, direction, connection), where direction refers to the port type which is either input or output, and connection refers to the connected ports of the upstream module or downstream module.
3. **Dataflow Graph Extraction**: Signal assignments and dependencies from the AST are analyzed to build a dataflow graph that represents how signals propagate within and across modules.
4. **Signal Chain Extraction**: AssertMiner leverages the MCG, I/O table, and data-flow graph to extract signal chains that trace inter-module signal propagation from top-level inputs to lower-level outputs. To guarantee robustness, internal module signals are pruned during chain simplification to prevent contamination from internal data flows which may include design bugs.

For example, Fig. 4 presents the AST data-flow graph and signal chain extraction results of `sha3` [34]. In the AST, only the signals and statements relevant to signal chain extraction are retained. Signal chains are generated through backward traversal of the data-flow graph. This process traces each output back to its input sources, forming concrete signal propagation paths that capture end-to-end dependencies within modules. After generation, the signal chains are further simplified by keeping only signals connected to module I/O ports. These simplified signal chains are then used in Step 2 to extract reliable module specifications.

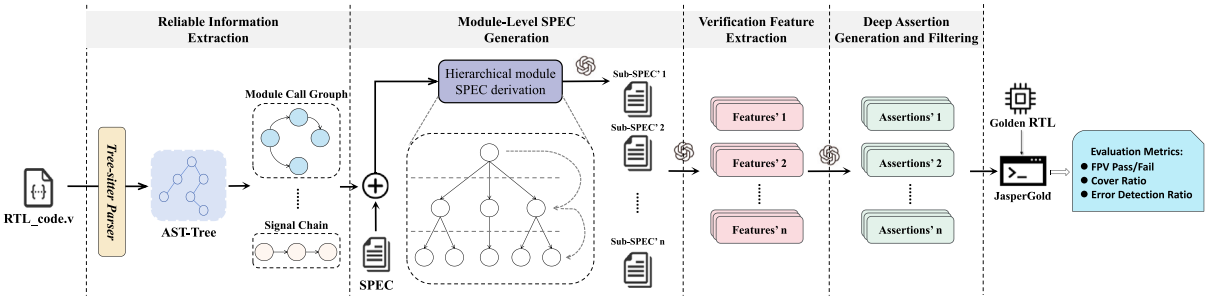

Fig. 3. The AssertMiner-pro framework first parses the RTL code under verification using the Tree-setter tool to generate an abstract syntax tree (AST). Reliable information is then extracted from the AST and, together with the original design specification file, provided as input to the LLM. Following a top-down hierarchical approach, module-level specifications are inferred for each module. Verification feature descriptions are subsequently extracted from each generated specification file to guide the LLM to produce deep assertions for the module's internal details. Finally, the correctness of these generated assertions is checked against the golden RTL, and relevant evaluation metrics are collected.

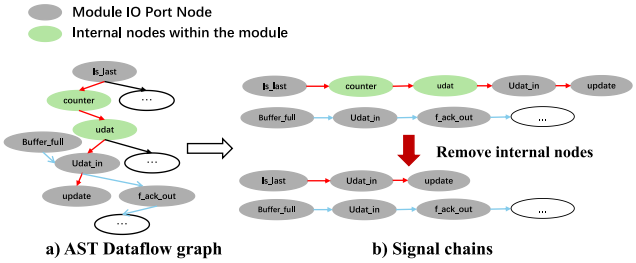

Fig. 4. Static structural information extracted from the RTL code of SHA3 [17].

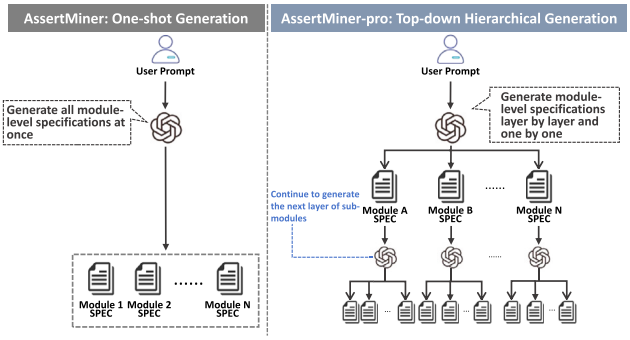

Fig. 5. The differences between AsserMiner and AssertMiner-pro during the generation of the module-level specification.

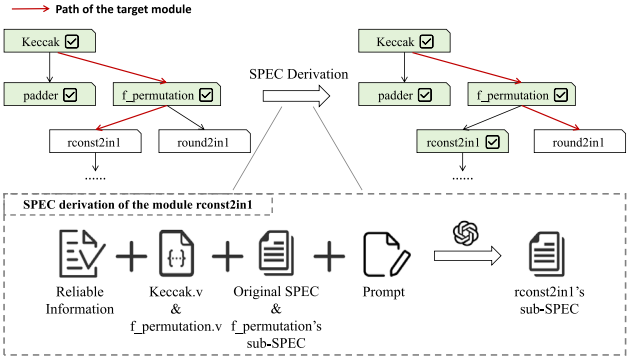

Fig. 6. Example of the derivation of the `rconst2in1` module-level specification in the SHA3 design. In this hierarchy, `Keccak` calls the `padder` and `f_permutation` modules, while `f_permutation` further calls the `rconst2in1` and `round2in1` modules.

4.3. Step 2: Top-down hierarchical derivation of module-level specifications

Due to the lack of module-level specifications, designers need to interpret and supplement their functionalities based on personal understanding. This uncertainty makes it difficult to directly generate assertions related to the micro-architecture. Moreover, the RTL implementation itself may contain design errors, making it an unreliable source for specification inference.

A natural idea is that if clear module-level specifications can be derived for each submodule, they can serve as a reliable foundation for generating micro-architecture-level assertions. As stated in Assumption 1, module input and output port configurations are typically assumed to be correct in practical verification scenarios. Based on Assumption 2, the code of other modules that call the target module can be included as auxiliary context. Even if these modules contain minor errors, their inclusion does not impede the LLM's ability to understand the target module's functional behavior. Therefore, we provide the LLM with the structural information extracted in Step 1, together with the code of all modules that call the target module. With these inputs, the LLM can infer the overall functionality of the target module and generate accurate module-level specifications by leveraging its reasoning capability. This idea is supported by [38], which shows that such inference is feasible.

Based on the analysis above, we first perform a hierarchical decomposition of modules according to their call relationships. We adopt a top-down approach to generate module-level specifications layer by layer. For each target module, the LLM is provided with four types of reliable information extracted in Step 1, the original top-level specification file, as well as the RTL code and generated specification files of all modules along the call path from the top-level module to the target module. Given these inputs, the LLM infers the functionality of the target module and generates a clear module-level specification. This process starts from the second layer of modules called by the top-level module and proceeds recursively through the hierarchy.

Fig. 5 illustrates the main differences between AssertMiner and AssertMiner-pro during the process of generating module-level specifications, and a concrete example is illustrated in Fig. 6. In the `sha3` design, `Keccak` serves as the top-level design, whose design description corresponds to the original `sha3` specification. Within `Keccak`, `padder` and `f_permutation` are instantiated. The `f_permutation` module further instantiates two submodules, `rconst2in1` and `round2in1`. The figure depicts an intermediate state in the top-down hierarchical generation of module-level specifications.

When generating the specification for the `rconst2in1` module, the LLM is provided with the following inputs: the reliable information extracted from the entire design, the RTL code and generated specification files of all modules along the call path (i.e., `Keccak`

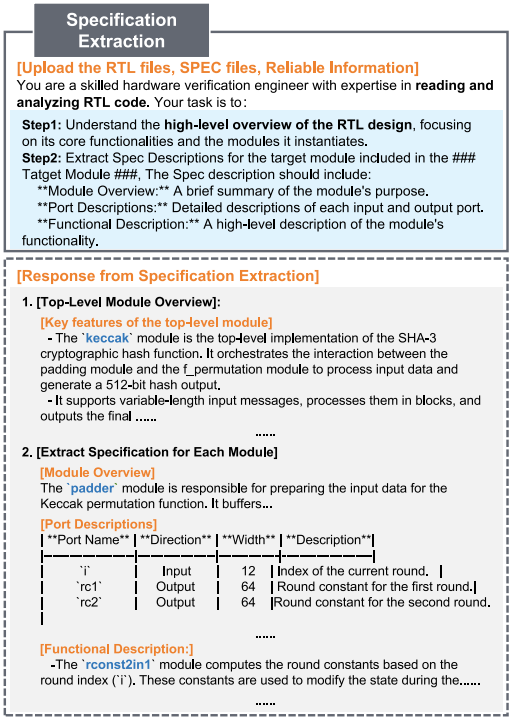

Fig. 7. Concise specification-extraction prompt and the generated specification file for the `rconst2in1` module.

and `f_permutation`), and the corresponding prompt. Leveraging its powerful reasoning capabilities, the LLM infers the overall functionality of the `rconst2in1` module as well as the functional behavior of its port signals, and generates the corresponding module-level specification file. Following the same top-down principle, the specification for the `round2in1` module is then generated in a similar manner, recursively repeating the procedure described above. The final concise specification-extraction prompt and the generated specification file for the `rconst2in1` module are illustrated in Fig. 7.

In AssertMiner, due to limitations in the LLM guidance strategy, deeply nested modules in the call hierarchy were often omitted, or the generated specifications contained insufficient semantic information. In contrast to AssertMiner, the proposed hierarchical strategy guides the LLM in a layer-by-layer manner to generate a dedicated specification file for each module. This targeted, top-down guidance enables more effective extraction of semantic information from deeply nested modules. A comparison of the specifications generated before and after introducing the hierarchical strategy is shown in Fig. 8. The AssertMiner framework misses two deeply nested submodules when generating specifications, whereas AssertMiner-pro is able to generate specifications for all called submodules.

4.4. Step 3: Verification feature extraction for modules

In practical top-down hardware development, there is often no explicit alignment between the overall specification and the detailed behaviors of submodules. To enable fine-grained and verifiable functional verification, verification engineers need to decompose module-level specifications into atomic verification features. These features are concise statements that capture specific properties or behaviors to be validated within each module. Based on the module-level specifications extracted in Step 2, we use the LLM to identify such verification features in a systematic manner. The model leverages the structural and functional information of the module to generate features that are precise and suitable for downstream assertion generation. This

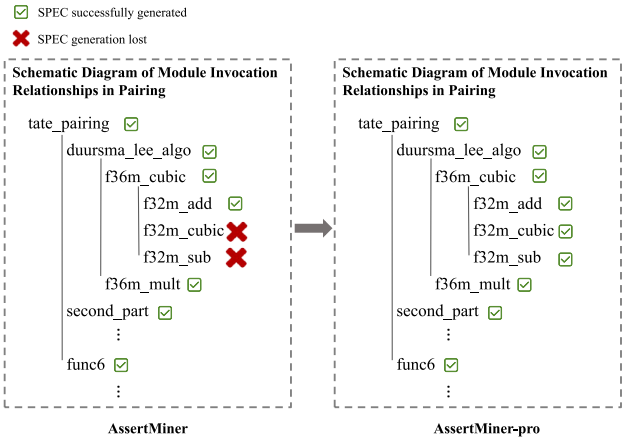

Fig. 8. Improved specification completeness for deep call-path modules using top-down hierarchical LLM guidance.

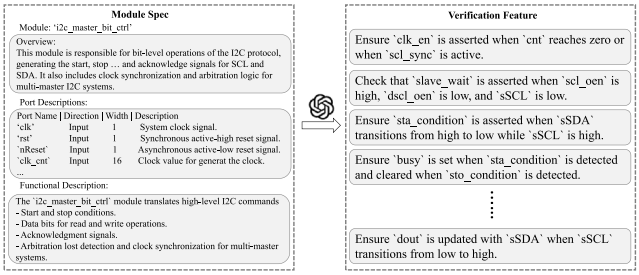

Fig. 9. Example of decomposing module's spec into verification features.

process improves coverage of module behaviors and ensures that each verification feature can be traced back to the original specification, thereby enhancing verification efficiency and reliability. Using the `padder` module of `sha3` as an example, the generated verification features are shown in Fig. 9, illustrating how higher-level specifications are decomposed into fine-grained verification items.

4.5. Step 4: Deep assertion mining

After obtaining the verification features from the previous steps, the next stage involves automatically mining deep assertions that capture the detailed signal behaviors within each module. These deep assertions serve as precise, machine-readable properties that monitor the internal state and interactions of module signals, enabling early detection of subtle functional deviations or corner-case bugs. To ensure consistency and interoperability among all mined assertions, we employ a prompting strategy that guides the LLM to produce outputs in a unified format, as shown below:

```
assert property (@(posedge <CLK>)
  (<AP> { && <AP> } { || <AP> } ...) |-> (<AP> {
       && <AP> } { || <AP> } ...)
);
```

where AP denotes an atomic proposition, which may consist of multiple signals formatted as `module.signal_name`, indicating the location of the signal within the module hierarchy.

This strategy not only standardizes the syntax and structure of the generated assertions but also facilitates downstream verification tasks, such as automated correctness checking and coverage analysis. By integrating verification features with structural and contextual information from the module, the LLM can generate assertions that are both comprehensive and targeted. These assertions capture the relationships

between internal signals while remaining traceable to the original specifications. The resulting deep assertions provide a solid foundation for functional verification. They achieve high coverage of internal module behavior and support efficient debugging and validation workflows.

4.6. Step 5: Assertion filtering

4.6.1. Analysis of incorrectly generated deep assertions

Further investigation reveals that a large portion of the incorrectly generated deep assertions by AssertMiner-pro originate from a common issue: incomplete information leads to the generation of incomplete assertions.

Specifically, in many cases, the available reliable information is sufficient for the LLM to infer the triggering conditions of an assertion, but insufficient to determine the corresponding outcomes. As a result, the generated assertions capture only one side of the property, yielding incomplete or invalid assertions. A concrete example is provided below:

```
assert property(@(posedge clk)disable iff(!reset)
  done |-> c0 == expected_c0 && c1 == expected_c1)
);
```

Many of the incorrectly generated assertions use placeholders such as expected_c0 or expected(c0) to represent portions of the information that were not available to the LLM. This indicates that the LLM could infer the triggering conditions of the assertion, but it lacks sufficient information to determine the correct outcomes. Consequently, these assertions are incomplete or invalid, as they fail to fully capture the intended property. The prevalence of such placeholders highlights that incomplete input information is the primary source of generation errors in deep assertions.

4.6.2. Filtering of incomplete assertions

To improve the quality of generated assertions, we further introduce an assertion filtering step to remove incomplete assertions generated under insufficient information conditions.

As discussed above, many incorrectly generated deep assertions contain placeholders such as `expected_c0`, indicating that the LLM can infer the triggering conditions of the property but cannot determine the corresponding expected outcomes. Such assertions are incomplete and cannot accurately represent the intended design behavior. Notably, this type of error is not rare: statistically, it accounts for approximately 23% of all incorrectly generated assertions on average, making it one of the dominant failure modes.

Therefore, during post-processing, AssertMiner-pro identifies assertions containing unresolved placeholders or undefined expected values and removes them from the final assertion set. This filtering strategy effectively eliminates a large portion of invalid deep assertions caused by incomplete information, thereby improving the overall correctness and reliability of the generated assertions.

5. Experimentation on coverage improvement with deep assertions

5.1. Experimental setup and evaluation metrics

The benchmark data used in this study are sourced from the open-source dataset in [34], while each design in this benchmark includes a specification file and the corresponding golden RTL implementation code. We selected eight circuits from the open-source dataset, chosen to represent a range of design scales and complexities, The detailed descriptions of these selected designs are presented in Table 1, where LoC-pre and LoC-Post denote the number of lines of code before and after synthesis, respectively, since the post-synthesis code lines and cell number better reflect the actual scale of the design. Since all assertions are evaluated using formal methods to obtain code coverage results, which inherently limits scalability, the selected circuits are already of substantial size for verification experiments.

Table 1
Summary of designs.

Circuit	LoC-Pre	LoC-Post	Cell Num.
sockit	1119	1056	364
I^2C	1282	5369	756
uart	1082	8730	1112
SHA3_2	498	77 427	16 585
SHA3	618	141 185	22 228
ECG	1635	398 686	59 084
tiny_pairing	1313	524 790	80 428
Pairing	2145	1561498	228 287

Table 2
Summary of evaluation metrics.

Evaluation metrics	Summary
N	The number of generated SVAs
S	The number of syntax-correct SVAs
P	The number of FPV-passed SVAs
BFC	Branch coverage in formal analysis
SFC	Statement coverage in formal analysis
TFC	Toggle coverage in formal analysis

*Note: The coverage metrics BFC, SFC and TFC are computed within the **Cones of Influence (COI)** [39] covered by the mined assertions, where a larger COI encompasses more signals, resulting in different denominators when the metrics are calculated. Therefore, in each design experiment, we use the number of signals involved in the largest COI as the standard denominator for the calculation.

We employ the commercial formal verification tool Cadence JasperGold (version: 21.12.002) for correctness analysis. All experiments are conducted on a server equipped with an Intel(R) Xeon(R) Gold 6148 CPU @ 2.40 GHz.

To evaluate the effectiveness of AssertMiner-pro, we conducted comparative experiments by selecting one pre-RTL assertion generation method and one RTL-stage method, both representing state-of-the-art (SOTA) approaches, and comparing them with the original AssertMiner framework under consistent experimental settings:

- **AssertLLM** [13]: This method is an assertion generation framework that leverages multiple specialized LLMs to translate unstructured natural-language specifications and waveform descriptions into SVAs capturing bit-width and functional properties.
- **AssertGen** [32]: This framework bridges the gap between high-level specifications and RTL designs by constructing cross-layer signal chains, thereby enabling the generation of RTL-stage assertions that cover both top-level and module-level signals.
- **AssertMiner** [17]: AssertMiner is an AST-guided framework that mines deep assertions by first extracting static structural information from RTL code, enabling LLMs to generate module-level specifications and assertions that capture micro-architectural behaviors overlooked by top-level approaches.

All three methods utilize GPT-4o as the experimental model to produce the results. We utilize the metrics provided in JasperGold, while the detailed evaluation criteria is shown in Table 2. In the experiment, we extract the correct assertions that pass FPV validation and use JasperGold once again to calculate the relevant coverage metrics with the correct assertions, and limit the FPV validation time as well as the COI measurement time to one hour.

This one-hour limit is chosen based on observations from previous work, such as the pioneering LLM-based assertion generation study AssertLLM [13], which used a five-hour timeout. Our experiments show that the vast majority of circuits complete within one hour, and for circuits that do not, the coverage-related metrics increase very slowly after the first hour. In industrial practice, it is common to terminate

Table 3
Performance comparison of the original AssertMiner framework and the improved AssertMiner-pro in mining deep assertions.

Circuit	AssertMiner				AssertMiner-pro			
	$N/S/P$	$BFC(\%)$	$SFC(\%)$	$TFC(\%)$	$N/S/P$	$BFC(\%)$	$SFC(\%)$	$TFC(\%)$
sockit	13/13/4	24.04	21.43	11.91	38/31/12	64.42	71.43	36.63
I^2C	28/28/12	79.51	81.05	80.21	36/35/16	79.51	82.26	81.60
uart	13/13/8	97.06	95.51	48.76	61/57/23	98.24	97.44	71.07
SHA3_2	14/14/10	90.48	93.10	81.44	60/52/31	85.71	89.66	81.83
SHA3	26/25/20	80.00	85.37	67.21	57/54/39	88.00	85.42	81.29
ECG	28/28/11	84.00	79.30	55.41	231/218/106	89.24	86.88	84.24
tiny_pairing	17/14/4	40.04	37.12	82.63	167/135/47	69.25	61.27	90.95
Pairing	28/24/13	84.96	74.93	52.08	439/390/167	91.15	92.84	79.88
Ave. Imp.	–	–	–	–	–	**+10.68**	**+12.42**	**+15.98**

Note: We attempt to guide AssertMiner to generate the same number of assertions as AssertMiner-pro. However, due to the absence of specifications for deeply invoked modules and inherent limitations of AssertMiner, a large portion of the generated assertions are redundant, resulting in no significant improvement in coverage metrics.

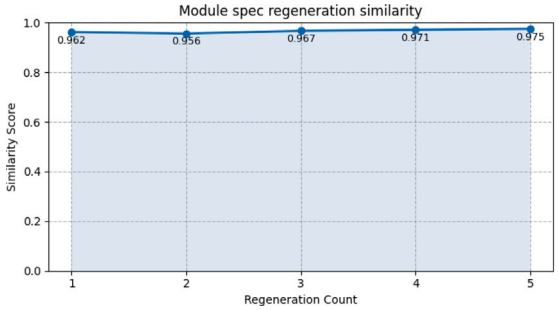

Fig. 10. Consistence of module spec regeneration.

validation when coverage growth becomes negligible to reduce verification time. Therefore, we adopt a one-hour limit as a practical standard for our experiments.

5.2. Module-level spec generation stability

To evaluate the stability of module-level specification generation, we regenerate the specification of the I^2C module five times under identical input conditions. We then measure the similarity between each regenerated specification and the initially generated specification using the Qwen3-Embedding model.

As shown in Fig. 10, The results demonstrate that the proposed framework maintains stable performance in the generation of module-level specification. Despite the inherent randomness of LLMs, the regenerated specifications remain highly consistent, with an average similarity above 95%.

5.3. Performance comparison of AssertMiner and AssertMiner-pro in deep assertion mining

As mentioned in Section 3.1, existing methods such as AssertLLM and AssertGen are not effective at generating deep assertions. To further investigate the performance of the original AssertMiner framework and the improved AssertMiner-pro in mining deep assertions, we conducted deep assertion mining on the selected circuits and reported only the coverage results of the generated deep assertions that are correct, as shown in Table 3.

The experimental results show that, compared to AssertMiner, AssertMiner-pro is able to mine a larger number of deep assertions targeting internal module behavior. Moreover, the number of mined deep assertions increases with the scale of the circuit design. This improvement primarily stems from the proposed top-down, hierarchical extraction strategy, which progressively propagates high-level design intent to lower-level modules and provides the LLM with richer structural and contextual information. As a result, the LLM is guided to explore more fine-grained functional details within complex designs.

The results further indicate that, across circuits of varying scales, most coverage metrics exhibit significant improvements with AssertMiner-pro. On average, the three evaluated coverage metrics each increase by more than 10 percentage points, demonstrating that the enhanced AssertMiner-pro framework can more comprehensively mine deep assertions and substantially improve verification coverage. Furthermore, as shown in Table 3, with the change in the generation strategy, a substantially larger number of deep assertions can be mined. However, this improvement comes at the cost of a reduced syntax-correct pass rate and FPV-passed rate of the generated assertions.

It should be noted that mining deep assertions is inherently a highly challenging task. The experimental results, along with the analysis of error sources, demonstrate that AssertMiner-pro possesses superior capability in extracting deep assertions compared to previous approaches. Looking ahead, if methods can be developed to provide LLM with more comprehensive and reliable information—without being affected by potential errors in RTL implementation—there is significant potential for further improvements in the correctness of generated deep assertions.

5.4. Overall design-level coverage evaluation

To validate AssertMiner-pro's efficacy, We integrated AssertMiner-pro with AssertLLM and AssertGen to observe coverage changes across these circuits. For a fair and comprehensive comparison, we also conducted the same integration using AssertMiner, the original version of our framework.

Tables 4 and 5 show the improvements in metrics for the two methods before and after integration with a set of deep assertions generated by AssertMiner and AssertMiner-pro. The results indicate that, when integrated with AssertMiner-pro, both methods achieve further and more significant coverage improvements compared to integration with the original AssertMiner in most circuits, and even in large circuits can still maintain a high improvement of coverage metrics. This observation suggests that AssertMiner-pro is more effective at mining deep assertions that exercise internal module behaviors and uncover previously unverified logic.

In contrast, for a small number of circuits where the improvements are marginal or negligible, further analysis reveals two primary reasons. First, these designs are relatively small or shallow in hierarchy, limiting the availability of meaningful deep internal behaviors that can be targeted by additional assertions. As a result, the potential coverage gain from deep assertions is inherently constrained. Second, some circuits already exhibit high baseline coverage due to the presence of simple control logic or well-constrained functionality, leaving limited room for further improvement. In such cases, additional deep assertions tend to overlap with existing verification checks and thus contribute little to measurable coverage gains.

Overall, these results demonstrate that AssertMiner-pro consistently outperforms the original AssertMiner in enhancing verification coverage, especially for large-scale designs with complex internal structures.

Table 4
Performance comparison of AssertLLM, AssertLLM + AssertMiner and AssertLLM + AssertMiner-pro across the selected circuits.

Circuit	AssertLLM			AssertLLM + AssertMiner			AssertLLM + AssertMiner-pro		
	$BFC(\%)$	$SFC(\%)$	$TFC(\%)$	$BFC(\%)$	$SFC(\%)$	$TFC(\%)$	$BFC(\%)$	$SFC(\%)$	$TFC(\%)$
sockit	3.8	2.38	0.67	24.04	21.43	11.86	64.42	71.43	46.76
I^2C	81.15	82.26	83.08	82.79	83.87	86.10	82.79	85.08	87.31
uart	97.65	93.75	34.55	97.65	96.15	62.83	98.24	97.44	79.45
SHA3_2	47.62	34.48	17.50	100	100	81.45	100	100	91.53
SHA3	92.00	90.24	66.74	100	95.37	76.28	100	100	91.02
ECG	84.02	80.94	67.40	84.22	80.94	73.45	89.24	86.91	83.06
tiny_pairing	58.96	52.41	79.44	61.61	59.46	84.46	69.25	67.28	91.01
Pairing	73.01	79.40	73.29	84.96	79.40	73.34	91.15	92.84	78.06
Ave. Imp.	–	–	–	**+12.13**	**+12.60**	**+15.89**	**+19.61**	**+23.14**	**+28.19**

Table 5
Performance comparison of AssertGen, AssertGen + AssertMiner and AssertGen + AssertMiner-pro across the selected circuits.

Circuit	AssertGen			AssertGen + AssertMiner			AssertGen + AssertMiner-pro		
	$BFC(\%)$	$SFC(\%)$	$TFC(\%)$	$BFC(\%)$	$SFC(\%)$	$TFC(\%)$	$BFC(\%)$	$SFC(\%)$	$TFC(\%)$
sockit	35.00	21.74	18.24	46.67	39.13	43.19	64.42	71.43	37.05
I^2C	82.61	82.26	79.86	82.61	82.32	81.14	83.34	87.32	84.03
uart	86.00	93.77	33.51	97.06	95.51	62.30	98.24	97.44	71.49
SHA3_2	85.70	68.97	62.16	100	100	81.30	100	100	81.32
SHA3	92.00	90.24	79.41	92.00	92.68	86.43	100	100	90.95
ECG	87.43	82.92	72.07	87.69	84.64	80.91	92.76	89.23	82.96
tiny_pairing	77.62	71.47	45.93	77.62	71.47	78.83	77.62	72.88	90.98
Pairing	79.61	62.27	53.15	85.80	83.13	71.42	91.15	93.55	84.98
Ave. Imp.	–	–	–	**+5.44**	**+9.41**	**+17.65**	**+10.20**	**+17.15**	**+22.55**

Table 6
Module-Level coverage comparison of AssertLLM and AssertLLM + AssertMiner-pro across selected modules.

Circuit	Module	AssertLLM			AssertLLM + AssertMiner-pro		
		$BFC(\%)$	$SFC(\%)$	$TFC(\%)$	$BFC(\%)$	$SFC(\%)$	$TFC(\%)$
I^2C	i2c_master_byte_ctrl	94.44	91.94	81.82	94.44	96.77	96.10
	i2c_master_bit_ctrl	84.75	88.36	95.19	88.14	91.10	97.12
SHA3_2	padder	42.86	33.33	3.70	100	100	98.72
	f_permutation	57.14	80.00	0.53	100	100	98.96
	rconst	/	0	0	/	100	35.23
	round	/	/	0	/	/	98.25
Pairing	f33m_mux2	/	/	7.41	/	/	36.89

Note: /indicates that JasperGold reports no available coverage data for the corresponding metric within the module. When a module is instantiated multiple times, only the results at the hierarchy level where it is first instantiated are counted.

Table 7
Module-Level Coverage Comparison of AssertGen and AssertGen + AssertMiner-pro across selected modules.

Circuit	Module	AssertGen			AssertGen + AssertMiner-pro		
		$BFC(\%)$	$SFC(\%)$	$TFC(\%)$	$BFC(\%)$	$SFC(\%)$	$TFC(\%)$
UART	uart_top	/	/	26.92	/	/	44.23
	uart_parser	55.86	50.38	30.08	96.55	94.66	81.20
SHA3_2	f_permutation	57.14	80.00	0.53	100	100	98.96
	rconst	/	0	0	/	100	35.23
	round	/	/	0	/	/	98.25
SHA3	f_permutation	71.43	60.00	97.92	100	100	97.94
ECG	f3m_inv	25.00	36.36	41.90	100	100	90.89
	f3m_mult	20.00	72.73	12.42	100	100	31.31
tiny_pairing	FSM	65.38	64.29	24.35	94.23	92.86	54.92
	PE	0	0	0.39	100	100	96.97
Pairing	duursma_lee_algo	90.91	63.64	15.08	90.91	90.91	53.77
	second_part	50.00	40.00	0.07	50.00	60.00	66.67
	f33m_neg	/	/	0	/	/	100
	f33m_inv	100	80.00	0.25	100	100	99.98
	f33m_mux2	/	/	0	/	/	30.78
	f3m_neg	/	/	0	/	/	87.11

5.5. Module-level coverage improvement analysis

To further analyze the impact of the generated deep assertions, we conducted an additional experiment to evaluate coverage improvements at the module level. Instead of reporting only overall design coverage, we measured the code coverage metrics of individual modules before and after incorporating the deep assertions generated by our method.

We picked out some modules that showed particularly significant improvements, as shown in Tables 6 and 7. The results indicate that

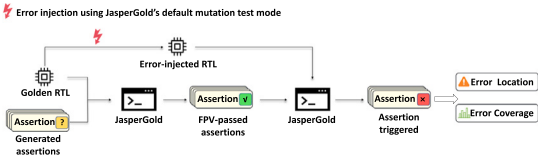

Fig. 11. Mutation testing process for assertion triggering.

incorporating the generated deep assertions leads to effective module-level coverage improvements. Compared to evaluating coverage only at the overall design level, the analysis reveals that deep assertions are particularly effective in improving branch, state, and signal toggle coverage within deeply nested modules. This improvement is more pronounced in complex submodules that exhibit rich internal control logic and signal interactions, where top-level assertions fail to adequately cover internal behaviors.

Furthermore, the coverage gains are not uniformly distributed across all submodules. Modules that are frequently activated along the design's execution paths or that implement critical functional logic benefit the most from deep assertions, while relatively simple or pass-through modules show marginal improvements. These observations indicate that deep assertions provide targeted verification benefits by directly constraining internal signal behaviors, thereby enhancing verification granularity and aiding in uncovering corner-case scenarios that are otherwise difficult to detect through top-level assertions alone.

6. Experimentation on error detection ratio via mutation testing

6.1. Experimentation setup and evaluation metrics

To further explore AssertMiner-pro's performance in mutation testing, we utilize the JasperGold tool to inject mutation errors into eight designs and perform error localization and coverage analysis, following the steps illustrated in Fig. 11.

To evaluate the effectiveness of deep assertions in error detection, we conduct mutation testing by integrating AssertMiner and AssertMiner-pro with existing assertion generation frameworks. For each target design, a set of mutants is automatically injected into the RTL using JasperGold. The evaluation metric used in this experiment is the *Error Detection Ratio (EDR)*, defined as the proportion of injected mutations that are successfully detected by at least one assertion. Formally, *EDR* is computed as the ratio of detected errors to the total number of injected mutants.

6.2. Evaluation of error detection ratio

We first measure the error detection ratio of the baseline methods (AssertLLM and AssertGen) using their originally generated assertions. We then augment these assertion sets with deep assertions generated by AssertMiner and AssertMiner-pro, respectively, and re-evaluate the error detection ratio under the same mutation configurations. Figs. 12 and 13 shows the error coverage improvements achieved by integrating AssertMiner and AssertMiner-pro with AssertLLM and AssertGen across six representative circuits. For each circuit, the baseline coverage is compared against the enhanced versions, demonstrating consistent and substantial improvements, particularly when using AssertMiner-pro.

The results indicate a substantial improvement in error detection ratio after integrating deep assertions generated by AssertMiner and AssertMiner-pro. Across all evaluated designs, both frameworks demonstrate a clear increase in the number of detected errors compared to the baseline assertion sets. Notably, AssertMiner-pro consistently achieves higher error detection ratios than the original AssertMiner, reflecting the benefits of its enhanced framework, which incorporates additional RTL implementation details and mitigates the impact of internal design errors. These findings confirm the effectiveness of deep assertions in capturing subtle and module-internal errors that are otherwise hard to detect by top-level assertions alone.

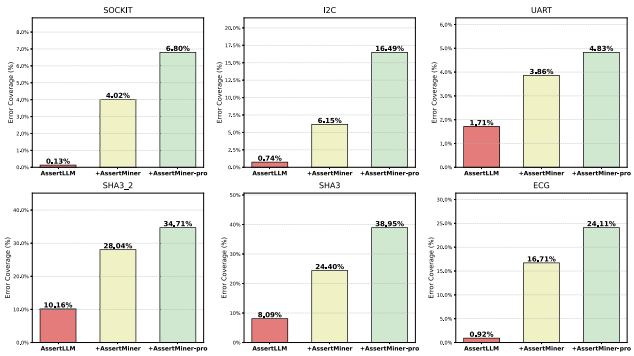

Fig. 12. Error coverage Improvement of AssertLLM Integrated with AssertMiner and AssertMiner-pro.
*Note: Due to the large scale of tiny_pairing and Pairing, mutation testing would require over 120 h; therefore, we exclude them from the experiment.
†Mutation testing is a highly constrained evaluation method. With a limited number of generated assertions, achieving high error coverage is challenging [17].

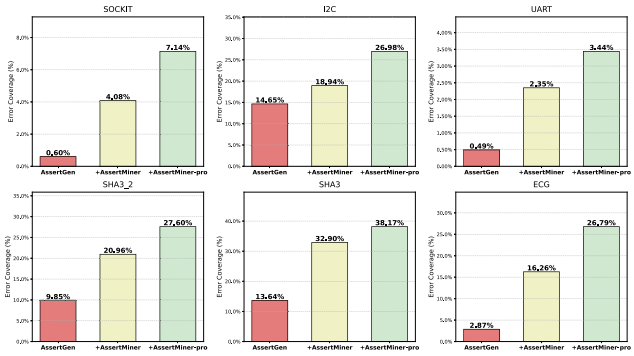

Fig. 13. Error coverage Improvement of AssertGen Integrated with AssertMiner and AssertMiner-pro.

7. Conclusion

This paper proposes AssertMiner-pro, an enhanced framework for deep assertion generation in hardware verification that integrates AST-based structural analysis with top-down hierarchical reasoning. By constructing accurate module-level specifications, AssertMiner-pro enables LLMs to generate a larger number of fine-grained assertions, improving error detection.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This paper is supported in part by the Chinese Academy of Sciences, China under grant No. XDB0660103, and in part by the National Natural Science Foundation of China (NSFC) under grant No. 92373206.

Appendix A. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.vlsi.2026.102797.

Data availability

Data will be made available on request.

References

[1] Vaishnavi Pulavarthi, Deeksha Nandal, Soham Dan, Debjit Pal, Assertionbench: A benchmark to evaluate large-language models for assertion generation, in: Findings of the Association for Computational Linguistics: NAACL 2025, 2025, pp. 8058–8065.

[2] Harry Foster, Part 1: The 2020 wilson research group functional verification study, 2021, https://Tinyurl.Com/Verification-Foster-Group.

[3] Yael Abarbanel, Ilan Beer, Leonid Gluhovsky, Sharon Keidar, Yaron Wolfsthal, Focs–automatic generation of simulation checkers from formal specifications, in: International Conference on Computer Aided Verification, Springer, 2000, pp. 538–542.

[4] Harry Foster, 2024 Wilson Research Group IC/ASIC functional verification trend report, https://Resources.Sw.Siemens.Com/Zh-TW/White-Paper-2024-Wilson-Research-Group-Ic-Asic-Functional-Verification-Trend-Report/.

[5] Hasini Witharana, Yangdi Lyu, Subodha Charles, Prabhat Mishra, A survey on assertion-based hardware verification, ACM Comput. Surv. 54 (11s) (2022) 1–33.

[6] Harry Foster, Assertion-based verification: Industry myths to realities (invited tutorial), in: International Conference on Computer Aided Verification, Springer, 2008, pp. 5–10.

[7] Yunsheng Bai, Ghaith Bany Hamad, Syed Suhaib, Haoxing Ren, Assertionforge: Enhancing formal verification assertion generation with structured representation of specifications and rtl, 2025, arXiv preprint arXiv:2503.19174.

[8] Amira A. Alshazly, Ahmed M. Elfatatry, Mohamed S. Abougabal, Detecting defects in software requirements specification, Alex. Eng. J. 53 (3) (2014) 513–527.

[9] Erik Seligman, Tom Schubert, M.V. Achutha Kiran Kumar, Formal Verification: an Essential Toolkit for Modern VLSI design, Elsevier, 2023.

[10] Rahul Kande, Hammond Pearce, Benjamin Tan, Brendan Dolan-Gavitt, Shailja Thakur, Ramesh Karri, Jeyavijayan Rajendran, (Security) assertions by large language models, IEEE Trans. Inf. Forensics Secur. 19 (2024) 4374–4389.

[11] Vaishnavi Pulavarthi, Deeksha Nandal, Soham Dan, Debjit Pal, Are LLMs ready for practical adoption for assertion generation? in: 2025 Design, Automation & Test in Europe Conference, DATE, IEEE, 2025, pp. 1–7.

[12] Chuyue Sun, Christopher Hahn, Caroline Trippel, Towards improving verification productivity with circuit-aware translation of natural language to systemverilog assertions, in: First International Workshop on Deep Learning-Aided Verification, 2023.

[13] Zhiyuan Yan, Wenji Fang, Mengming Li, Min Li, Shang Liu, Zhiyao Xie, Hongce Zhang, Assertllm: Generating hardware verification assertions from design specifications via multi-llms, in: Proceedings of the 30th Asia and South Pacific Design Automation Conference, 2025, pp. 614–621.

[14] Hafiz Abdul Quddus, Md Sanowar Hossain, Ziya Cevahir, Alexander Jesser, Md Nur Amin, Enhanced VLSI assertion generation: Conforming to high-level specifications and reducing LLM hallucinations with RAG, in: DVCon Europe 2024; Design and Verification Conference and Exhibition Europe, VDE, 2024, pp. 57–62.

[15] Fnu Aditi, Michael S. Hsiao, Hybrid rule-based and machine learning system for assertion generation from natural language specifications, in: 2022 IEEE 31st Asian Test Symposium, IEEE, 2022, pp. 126–131.

[16] Matthias Cosler, Christopher Hahn, Daniel Mendoza, Frederik Schmitt, Caroline Trippel, Nl2spec: Interactively translating unstructured natural language to temporal logics with large language models, in: International Conference on Computer Aided Verification, Springer, 2023, pp. 383–396.

[17] Hongqin Lyu, Yonghao Wang, Jiaxin Zhou, Zhiteng Chao, Tiancheng Wang, Huawei Li, AssertMiner: Module-level spec generation and assertion mining using static analysis guided LLMs, in: 34th Asian Test Symposium, IEEE, 2025.

[18] Fenghua Wu, Evan Pan, Rahul Kande, Michael Quinn, Aakash Tyagi, David Kebo Houngninou, Jeyavijayan Rajendran, Jiang Hu, Spec2Assertion: Automatic pre-RTL assertion generation using large language models with progressive regularization, 2025, arXiv preprint arXiv:2505.07995.

[19] Hongqin Lyu, Yunlin Du, Yonghao Wang, Zhiteng Chao, Tiancheng Wang, Huawei Li, AssertFix: Empowering automated assertion fix via large language models, 2025, arXiv preprint arXiv:2509.23972.

[20] Hubert Kaeslin, Top-down digital VLSI Design: from Architectures to Gate-Level Circuits and FPGAs, Morgan Kaufmann, 2014.

[21] F Sforza, L Battu, M Brunelli, A Castelnuovo, M Magnaghi, A" design for verification" methodology, in: Proceedings of the IEEE 2001. 2nd International Symposium on Quality Electronic Design, IEEE, 2001, pp. 50–55.

[22] Amir Hekmatpour, Azadeh Salehi, Block-based schema-driven assertion generation for functional verification, in: 14th Asian Test Symposium, IEEE, 2005, pp. 34–39.

[23] Shobha Vasudevan, David Sheridan, Sanjay Patel, David Tcheng, Bill Tuohy, Daniel Johnson, Goldmine: Automatic assertion generation using data mining and static analysis, in: 2010 Design, Automation & Test in Europe Conference & Exhibition (DATE 2010), IEEE, 2010, pp. 626–629.

[24] Alessandro Danese, Nicolò Dalla Riva, Graziano Pravadelli, A-team: Automatic template-based assertion miner, in: Proceedings of the 54th Annual Design Automation Conference 2017, 2017, pp. 1–6.

[25] Samuele Germiniani, Graziano Pravadelli, Harm: a hint-based assertion miner, IEEE Trans. Comput.-Aided Des. Integr. Circuits Syst. 41 (11) (2022) 4277–4288.

[26] Takamitsu Yamada, Assertion generating system, program thereof, circuit verifying system, and assertion generating method, 2009, US Patent 7, 603, 636.

[27] Junchen Zhao, Ian G. Harris, Automatic assertion generation from natural language specifications using subtree analysis, in: 2019 Design, Automation & Test in Europe Conference & Exhibition, DATE, IEEE, 2019, pp. 598–601.

[28] Ganapathy Parthasarathy, Saurav Nanda, Parivesh Choudhary, Pawan Patil, SpecToSVA: Circuit specification document to systemverilog assertion translation, in: 2021 Second Document Intelligence Workshop At KDD, 2021.

[29] Mohammad Reza Heidari Iman, Gert Jervan, Tara Ghasempouri, Artmine: Automatic association rule mining with temporal behavior for hardware verification, in: 2024 Design, Automation & Test in Europe Conference & Exhibition, DATE, IEEE, 2024, pp. 1–6.

[30] Karthik Maddala, Bhabesh Mali, Chandan Karfa, Laag-rv: Llm assisted assertion generation for rtl design verification, in: 2024 IEEE 8th International Test Conference India (ITC India), IEEE, 2024, pp. 1–6.

[31] Bhabesh Mali, Karthik Maddala, Vatsal Gupta, Sweeya Reddy, Chandan Karfa, Ramesh Karri, Chiraag: Chatgpt informed rapid and automated assertion generation, in: 2024 IEEE Computer Society Annual Symposium on VLSI, ISVLSI, IEEE, 2024, pp. 680–683.

[32] Hongqin Lyu, Yonghao Wang, Yunlin Du, Mingyu Shi, Zhiteng Chao, Wenxing Li, Tiancheng Wang, Huawei Li, AssertGen: Enhancement of LLM-aided assertion generation through cross-layer signal bridging, in: 31th Asia and South Pacific Design Automation Conference, IEEE, 2026.

[33] Zhiteng Chao, Yonghao Wang, Xinyu Zhang, Jiaxin Zhou, Tenghui Hua, Husheng Han, Tianmeng Yang, Jianan Mu, Bei Yu, Rui Zhang, et al., Think with self-decoupling and self-verification: Automated RTL design with backtrack-ToT, in: 2026 Design, Automation & Test in Europe Conference, IEEE, 2026, pp. 1–7.

[34] IWLS 2005 Benchmaeks, https://Iwls.Org/Iwls2005/Benchmarks.Html.

[35] Shang Liu, Wenji Fang, Yao Lu, Qijun Zhang, Hongce Zhang, Zhiyao Xie, Rtlcoder: Outperforming gpt-3.5 in design rtl generation with our open-source dataset and lightweight solution, in: 2024 IEEE LLM Aided Design Workshop, LAD, IEEE, 2024, pp. 1–5.

[36] YunDa Tsai, Mingjie Liu, Haoxing Ren, Rtlfixer: Automatically fixing rtl syntax errors with large language model, in: Proceedings of the 61st ACM/IEEE Design Automation Conference, 2024, pp. 1–6.

[37] Ke Xu, Jialin Sun, Yuchen Hu, Xinwei Fang, Weiwei Shan, Xi Wang, Zhe Jiang, Meic: Re-thinking rtl debug automation using llms, in: Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design, 2024, pp. 1–9.

[38] Mengming Li, Wenji Fang, Qijun Zhang, Zhiyao Xie, Specllm: Exploring generation and review of vlsi design specification with large language model, in: 2025 International Symposium of Electronics Design Automation, ISEDA, IEEE, 2025, pp. 749–755.

[39] JasperGold: the Next Generation, https://community.cadence.com/cadence_blogs_8/b/breakfast-bytes/posts/jasper-ml.